Zewail City of Science and Technology

3D Unity Project: SWGCG351, Game Design & Dev

Project Title: Eclipse Protocol
Game Genre: Top-Down Sci-Fi Survival Strategy
Team size: 1-2 (Max is 2 students)
Course Grades Weight: 15 Grades

Project Objective:

The primary goal of this project is the successful migration and professionalization of the *Eclipse Protocol* codebase from a 2D procedural framework (Love2D) to a **3D component-based architecture** using the **Unity Engine**. This transition necessitates a shift from manual coordinate-based scripts to a modular, data-driven system leveraging 3D Physics, NavMesh Navigation, and C# Scripting. Beyond technical execution, the project integrates industry-standard pre-production by requiring a comprehensive 10-page Game Design Document (GDD). This ensures that every 3D mechanic, from spatial player movement to complex AI state machines, is grounded in clear technical design before implementation.

The Project Emphasizes:

- **3D Architectural Scalability:** Replacing 2D global tables with encapsulated C# components, ScriptableObjects for data management, and Unity Prefabs for procedural 3D environment construction.
- **Spatial Engineering:** Utilizing the Unity Editor for 3D asset pipeline management, NavMesh for intelligent pathfinding, and 3.5D Camera systems.
- **Documentation-Driven Development:** Bridging the gap between conceptual 3D design and technical deployment through a mandatory 10-page "Technical Bible."
- **Immersive Polish:** Leveraging the Universal Render Pipeline (URP) for 3D lighting, the Animator Window for 3D state transitions, and the AudioMixer for 3D spatialized feedback.

Important Tip: Make sure to sign up for the **Unity Student Plan**. It gives you free access to:

1. **Unity Pro** (the paid version of the editor).
2. **Asset Store credits** or discounts.

An imaginary view of how the built game may look & feel

This project is an opportunity to build a professional 3D piece for your portfolio, something you can truly be proud to feature on your resume. While the provided visuals represent the quality and aesthetic target we are aiming for, we encourage you to use your imagination. Your creativity is highly recommended; feel free to put your own unique spin on the 'Eclipse Protocol' universe.

Project Phases Timeline

Project Phases	Deliverables	Grade (%)	Deadline
Phase-1	Midway Delivery	**30%**	Sunday 26/4/2026
Final Phase	Final Delivery	**70%**	Sunday 3/5/2026
Bonus	Bonus	**5 Grades**	TBD

I. Game Design Document (GDD) Requirements:

The GDD serves as the technical and creative blueprint for the project. It must be a **10+ pages professional document** (PDF format) covering the following:

Page 1: Project Identity & Executive Summary

- **Project Branding:** Final title, logo/typography style, and contact info.
- **Version History:** A table tracking document iterations (e.g., v1.0, v1.1).
- **High Concept:** A one-sentence pitch of the 3D survival experience.
- **Design Pillars:** The 3 core values (e.g., *Tension, Scarcity, Fluidity*).
- **Target Audience:** Who is playing and what is the intended "Form of Fun" (e.g., *Challenge* and *Discovery*)?

Page 2: 3D Vision & Core Gameplay Loop

- **The 3D Vision:** Description of how the Z-axis and 3D perspective enhance the "failing station" atmosphere.
- **Core Loop Diagram:** A visual flow of *3D Exploration* → *Resource Collection* → *Node Repair* → *Survival/Evacuation*.
- **Camera Perspective:** Technical details on the 3D camera (e.g., Top-Down Orthographic vs. Perspective with a 60 tilt).

Page 3: 3D Player Mechanics & Controls

- **3D Input Mapping:** Table of controls (WASD for X/Z movement, Space for Dash, Mouse for aiming/interaction).
- **Rigidbody (3D) Implementation:** Technical specs for the Player's **Capsule Collider** and **Rigidbody** settings (Mass, Drag, Constraints).
- **Movement Logic:** Mathematical breakdown of how velocity is applied in 3D space to avoid "sliding" or "moon-walking."

Page 4: The 3D Dash & Survival Systems

- **The 3D Dash:** Calculation of the dash vector. Does the robot dash toward the mouse cursor or the movement direction?
- **I-Frame Logic:** How the **3D Trigger** system ignores enemy damage during the dash duration.
- **Resource Formulas:** Detailed math for **Energy depletion** (cost per dash) and **Health regeneration** (if applicable).

Page 5: 3D AI & NavMesh Navigation

- **NavMesh Architecture:** Description of the "Baking" process, how the AI understands walkable surfaces vs. 3D obstacles.
- **AI State Machine (FSM):** A detailed flowchart showing transitions between *Idle, Patrol (Waypoints), Chase (Player Target), and Return*.

- **3D Detection:** Use of Physics.OverlapSphere or **Raycasting** for 3D Line-of-Sight (ensuring drones don't see through 3D walls).

Page 6: Procedural 3D Environment Generation

- **3D Room Prefabs:** A catalog of 3D "tiles" or "modules" (e.g., L-junctions, T-junctions, Hubs) with their dimensions.
- **The Spawner Algorithm:** Step-by-step logic of the **Procedural Spawner**:
    1. Select a random module.
    2. Check for 3D overlap.
    3. Snap to the "Exit Anchor" of the previous room.
- **Cleanup Logic:** How the game handles the destruction of 3D geometry to maintain 60 FPS.

Page 7: Technical Architecture (C# & Unity)

- **Class Hierarchy:** List of core scripts (e.g., PlayerController.cs, NavAI.cs, ProceduralLevelGen.cs).
- **ScriptableObjects:** Specification of 3D data containers (e.g., DroneStats.asset containing speed, health, and detection radius).
- **Layer Collision Matrix:** A table defining which 3D layers (Player, Enemy, Wall, Bullet) interact.

Page 8: 3D UI & HUD Design

- **Screen-Space vs. World-Space UI:** Will the health bar follow the robot in 3D space or stay fixed on the screen?
- **Canvas Layout:** Mockup of the 3D HUD (Energy Sliders, Mini-map/Compass, Mission Log).
- **Responsive Scaling:** How the UI handles different 3D aspect ratios (16:9 vs 21:9).

Page 9: 3D Visuals, Lighting, & Animation

- **Mecanim Animator:** A screenshot or diagram of the **3D Animator Controller** (Idle → Walk → Dash → Death).
- **URP Lighting Plan:** Placement of **Point Lights** and **Spotlights** within 3D room prefabs to create a moody atmosphere.
- **VFX Graph:** List of 3D particle effects (e.g., sparks from failing wires, drone explosion shards).

Page 10: 3D Spatial Audio & Roadmap

- **AudioMixer Routing:** Technical setup for **Master, Music, and SFX** groups.
- **3D Spatial Blend:** Description of the attenuation curves, how sound drops off over a 3D distance (MaxDistance vs MinDistance).
- **Final Milestones:** Timeline from the **Phase 1 3D Prototype** to the **Final Phase - Full Game**.

II. Unity 3D Technical Requirements: Detailed Breakdown

1. Player & 3D Physics Systems

In a 3D Unity environment, the player must interact with a three-dimensional volume rather than a flat plane.

- **3D Rigidbody Component:** Movement must be driven by Rigidbody.linearVelocity or Rigidbody.AddForce within the FixedUpdate() method. This ensures that movement remains consistent regardless of the frame rate.
- **3.5D Camera (Top-Down):** The camera should be positioned at a high angle (e.g., Y=10, RotationX=60). Use Cinemachine or a SmoothDamp script to prevent "jittery" movement as the player navigates the station.
- **The 3D Dash (Vector Physics):** The dash is a physics-based impulse.
    - **Logic:** When the dash is triggered, the script should apply a high-velocity burst in the direction of the player's current movement vector.
    - **I-Frames:** Utilize a Layer-based Collision change or a Boolean flag during the dash duration to ignore triggers from enemy CapsuleColliders. In Unity, a CapsuleCollider is the standard shape used to wrap around a human-shaped or robot-shaped 3D model.
- **Physics Layer Matrix:** Students must define specific layers: Player, Enemy, Wall, and Projectile. In the **Project Settings**, they must disable collisions between Enemies (if desired) and ensure that Projectiles interact only with intended targets.

2. 3D Navigation & AI (The NavMesh System)

3D AI does not move in straight lines; it finds the most efficient path through the environment.

- **NavMesh Agent Component:** Every drone must have a NavMeshAgent. Instead of calculating x + 1, the code will use agent.SetDestination(target.position).
- **Baking the Environment:** The procedural room prefabs must be marked as **Navigation Static**. The NavMesh must be "baked" so drones understand where floors end and walls begin.
- **State-Based AI (FSM):**
    - **Patrol State:** The agent cycles through a List<Transform> of waypoints placed within the 3D room.
    - **Chase State:** Triggered when Vector3.Distance between the player and the drone is less than the detectionRange. The drone abandons its path and recalculates a route to the player's real-time X, Y, Z coordinates.
- **3D Detection (Proximity & Vision):** Use Physics.OverlapSphere to create a 3D "detection bubble" around the drone.

3. Procedural 3D Environment & Spawning

The station is no longer a grid of sprites; it is a collection of 3D modules.

- **Prefab Room Modules:** Each room is a 3D Prefab containing a floor mesh, wall meshes, and "Anchor Points" (Empty GameObjects).
- **The Spawning Algorithm:** 1. When the player enters a **Door Trigger** (a BoxCollider with isTrigger enabled), the LevelManager picks a random Prefab from the Room Pool.
  2. The new room is instantiated and its "Entrance Anchor" is snapped to the current door's "Exit Anchor."
- **Dynamic Lighting:** Each room prefab should include **Point Lights** or **Spotlights**. To maintain performance, use **Real-time Shadows** sparingly, or rely on **Baked Lighting** if the rooms are static.
- **Object Pooling:** For high-frequency objects like 3D projectiles or sparks, implement an **Object Pool** to avoid the CPU overhead of frequent Instantiate and Destroy calls.

4. UI, Audio, & 3D Polish

The 3D model provides the "visuals," while the CapsuleCollider provides the "physical body" that hits walls and takes damage. You cannot just move a static box. You must have a 3D character (like a robot or astronaut) that has bones (a rig). Your 3D model needs specific animations for when the player hits the "Dash" button or when their health hits zero. A cube can't "play a death sequence," but a 3D character model can collapse or explode.

- **3D Spatial Audio:** Unlike Love2D's global sound, Unity sounds must have a **3D Spatial Blend** of 1.0.
  - **Attenuation:** The "Rolloff" curve must be set so the drone's hum grows louder as the player approaches.
- **AudioMixer Groups:** Route all 3D sound effects to an SFX group and the background music to a Music group. Use an **Audio Low Pass Filter** when the game is paused to create a "muffled" effect.
- **The Unity Animator (Mecanim):** 3D models require an **Animator Controller**.
  - Create a **Blend Tree** for the robot's movement (transitioning smoothly from *Idle* to *Walk* based on a magnitude parameter).
  - Use **Animation Triggers** for the Dash and Death sequences.
- **Visual Feedback (VFX):** Use the **Particle System** for 3D feedback, such as a "Flash" when the player takes damage or "Steam" emitting from damaged power nodes.

III. Project Phases & Deliverables:

Phase 1: Midway Delivery (30%):

I. The 10-Page Technical GDD (Full Document)

At this stage, the GDD must be **100% complete**. It is not a draft; it is the technical manual the student will follow to finish the game.

- **Architecture Mapping:** Must define all C# class names and their relationships.
- **Physics Constants:** Explicitly state the Rigidbody mass, drag, and Dash force values.
- **AI Logic:** Must include a visual flowchart of the Finite State Machine (FSM).
- **UI Mockups:** Precise layout of where HUD elements will sit in 3D space.

II. 3D Player & Physics Prototype

The player must be able to navigate a 3D environment with "snappy" and professional-feeling controls.

- **Rigidbody-Based Movement:** No "teleporting" (transform.position) movement. Must use AddForce or velocity in FixedUpdate to allow for proper 3D collisions with walls.
- **The Dash Mechanic:** A functional 3D dash burst. It must consume a placeholder "Energy" variable and have a functioning cooldown timer.
- **3.5D Camera Follow:** The camera must be positioned at a fixed high-angle offset, following the player smoothly without clipping through the 3D geometry.
- **Collision Layers:** Separation of the "Player" layer and "Environment" layer to prevent the player from walking through 3D walls.

III. 3D AI & Navigation (The NavMesh Test)

This is the most critical technical hurdle of Phase 1.

- **Baked NavMesh:** A single 3D test room with a "Baked" Navigation Mesh.
- **The Patrol Drone:** At least one drone prefab with a NavMeshAgent.
- **Waypoint Logic:** The drone must automatically move between a minimum of two 3D waypoints (A to B to A) using the NavMesh to avoid obstacles.
- **Proximity Trigger:** The drone must correctly identify when the player enters its 3D "Detection Sphere" (even if it doesn't "chase" yet, it must log the detection in the console).

IV. Static 3D Test Environment

While procedural generation is a Phase 2 requirement, Phase 1 requires a "Sandbox" to test the systems.

- **3D Geometry:** A manually placed room using 3D meshes (Floors, Walls, Pillars).
- **Basic URP Lighting:** At least one Directional Light and two Point Lights to demonstrate the sci-fi aesthetic.
- **Prefab Setup:** The Player, Drone, and Energy Cell must be saved as Unity Prefabs to ensure they are ready for the procedural spawner in the final phase.

Final Phase: Final Delivery (70%)

I. Procedural Level Generation (The Technical Core)

The manual sandbox from Phase 1 must be replaced with a dynamic system.

- **The Room Spawner:** You must implement a script that randomly instantiates 3D Room Prefabs from a defined list.
- **Grid or Node Logic:** Rooms must snap together without overlapping. You should use a "Seed" system to ensure levels can be replicated or randomized.
- **Dynamic NavMesh:** Since the level changes every time, you must use **NavMesh Components** (specifically `NavMeshSurface`) to "Bake" the navigation data at runtime so the AI knows where to walk in the new layout.

II. Advanced AI: The "Hunt" State

The Patrol Drone from Phase 1 must now evolve into a fully functioning threat using the FSM defined in your GDD.

- **State Transition:** When the player enters the "Detection Sphere," the Drone must switch from `Patrol` to `Chase`.
- **Pathfinding Pursuit:** The Drone must use its `NavMeshAgent` to calculate the shortest path to the player's 3D coordinates.
- **Attack Logic:** If the Drone gets within a specific "Attack Range," it must trigger a combat behavior (e.g., a projectile shot or a physical dash at the player).

III. Game Loop & UI Systems

This phase transforms a "prototype" into a "game" by adding win/loss conditions.

- **The Energy Economy:** The "Energy" variable from Phase 1 must now be fully integrated. Dashing consumes energy; picking up **Energy Cell Prefabs** restores it.
- **Win/Loss States:** * **Loss:** Health or Energy reaches zero (triggering a "Game Over" screen).
    - **Win:** Reaching a specific "Extraction Point" prefab at the end of the procedural level.
- **The HUD (Heads-Up Display):** Implementation of the UI Mockups from your GDD using **Unity UI (Canvas)**. This must include real-time bars for Health, Energy, and a Cooldown icon for the Dash.

IV. Technical Polish & Optimization

The final 70% of the grade relies heavily on the "feel" and stability of the build.

- **VFX & Feedback:** Implementation of **Particle Systems** for the Dash mechanic and Drone destruction.
- **Camera Shake:** Adding a subtle "Screen Shake" script when the player takes damage or performs a high-velocity dash.
- **URP Post-Processing:** Use the Universal Render Pipeline to add **Bloom** (for glowing sci-fi lights) and **Vignette** to enhance the atmosphere.

- **The Final Build:** A stable, standalone executable (.exe or .app). The console must be free of "NullReference" errors during a full playthrough.

Important Rules:

1. For each phase, **if a late submission is permitted, a 25% penalty will be incurred** for each day after the deadline. After two overdue days, no submissions will be accepted.
2. Each phase's deliverables must be uploaded to the **Google Classroom** assigned assignment and pushed to the repository **before the submission deadline**.
3. The deliverable version uploaded to the **Google Classroom** assignment is the one that will be evaluated. If it's a team project, the team representative can just upload the deliverable for other members. The GitHub version will serve only as a reference to track progress.
4. **Version Control:** All projects must use GitHub to track progress. Commits should reflect contributions from all members. **GitHub is mandatory in the evaluation process starting Phase-1** and must follow the "GitHub Repository Structure" mentioned. Failing to maintain an updated GitHub repository for your projects may result in a penalty of **up to 20%** for the phase under evaluation.
5. **Documentation Requirements**
    a. Each phase must include clear and complete documentation as outlined in the tasks, if any.
6. **Although this is a team project, every member is expected to fully understand all submitted work. TAs may ask any student about any part of the deliverable** (diagrams, code, documentation, etc). Any student unable to explain their project/code may lose marks individually, even if the deliverable was submitted.
7. **Cheating** could result in a negative grade (up to - 10 Course Grades).
8. **Academic Integrity Reminder**: Directly copying or pasting content from AI tools (such as ChatGPT or others) into your course deliverables is considered academic misconduct (Up to cheating). Your submitted work should reflect your own effort, writing style, creativity, and uniqueness. You may use AI tools to support your learning or improve your understanding, but the final content must be genuinely your own. All submissions/deliverables are subject to plagiarism and AI-detection checks. If there is reasonable suspicion of AI-generated or copied content, the lecturer/TA reserves the right to:
    - Assign a penalty for **cheating** if misuse is confirmed.
    - Request additional work or conduct an oral examination to verify authorship and understanding of the submitted material.

9. Each phase submission (when needed) must include **clear documentation** as required.

GitHub Repository & Version Control Requirement

- ➔ Each team must create and maintain a dedicated GitHub repository for this project. All development work must be committed regularly throughout the project period. The repository will be used to track progress, evaluate development consistency, and verify individual contributions.
- ➔ Teams are expected to make meaningful, incremental commits that reflect continuous development rather than uploading the entire project at once. Commit messages should be clear and descriptive. The repository must include a well-organized folder structure, source code, assets (where applicable), and a README file explaining how to run the game.
- ➔ Failure to demonstrate a consistent development history through GitHub commit activity may impact the project evaluation.

Grading Policy

- Understanding deliverables and their details will be evaluated individually for each team member.
- Your project GitHub is part of the evaluation weight.
- All the mentioned requirements in each phase are part of the evaluation criteria weights.

Draft: Grading Distribution Example

- Phase 1 – 30%
- Final Phase – 50%
- Code Quality & Architecture – 10%
- Creativity & Polish – 10%

Deliverable Discussion/Response Policy:

- If a student is asked a question during evaluation and does not answer, **5 points will be deducted**.
- If the student fails to answer a second question, **an additional 5 points will be deducted**.

- If the student fails to answer a third question, **the student will receive 0 for the Individual Understanding mark**.
- If the student fails to answer a fourth question, the student will receive 0 for the deliverable itself, regardless of the team's mark

Project Deliverable Documents

All project deliverables should have the following cover page:
Zewail City of Science, Technology and Innovation
University of Science and Technology
School of Computational Sciences and Artificial Intelligence

SWGCG351- Spring 2026

Game Development and Design
<Project Name>

- Student#1 <Name> <ID>
- Student#2 <Name> <ID>

<Date>